

Made in Blender.

STUDYING
CS AND FINANCE, WATERLOO

SITE
DANIELWLIU.COM

GITHUB
DANIELWLIU07

LINKEDIN
IN/DANIELWLIU

LINES OF CODE **95,500** | BUILD TIME **36 HRS** | TEAM **4 PEOPLE**

An undo button for your room.

A robot that keeps your room in git, the tool programmers use to save versions of their code. It notices when things move, plans how to put them back, and when something goes wrong it **files its own bug report**. Built in 36 hours because the idea would not leave me alone.

MADE AT
Hack the North 2026

TEAM
Four



THE ROBOT, CHECKING THE ROOM



WHAT THE ROBOT SEES: TWO CAMERAS, LIKE TWO EYES, FOR DEPTH



THE ROOM IN 3 CM CUBES

HOW IT WORKS

SEEING

A map made of 3 cm cubes

The robot builds its own 3D map from two cameras. Anything rising 9 cm above the floor becomes an object. An image AI names the objects it knows. The rest stay "unknown" instead of getting a guess.

NOTICING

Seen twice before it counts

Each scan works like git checking for unsaved changes. A move only counts once two separate scans of that spot agree, so a hand passing through is never logged as a move.

UNDOING

A plan that never collides

Undo becomes an ordered tidy-up plan that never puts an object down where another one sits. If two objects trade places, one is set aside first. The real robot drove to each object and pointed. Picking up ran in simulation.

Every save is a snapshot in git.

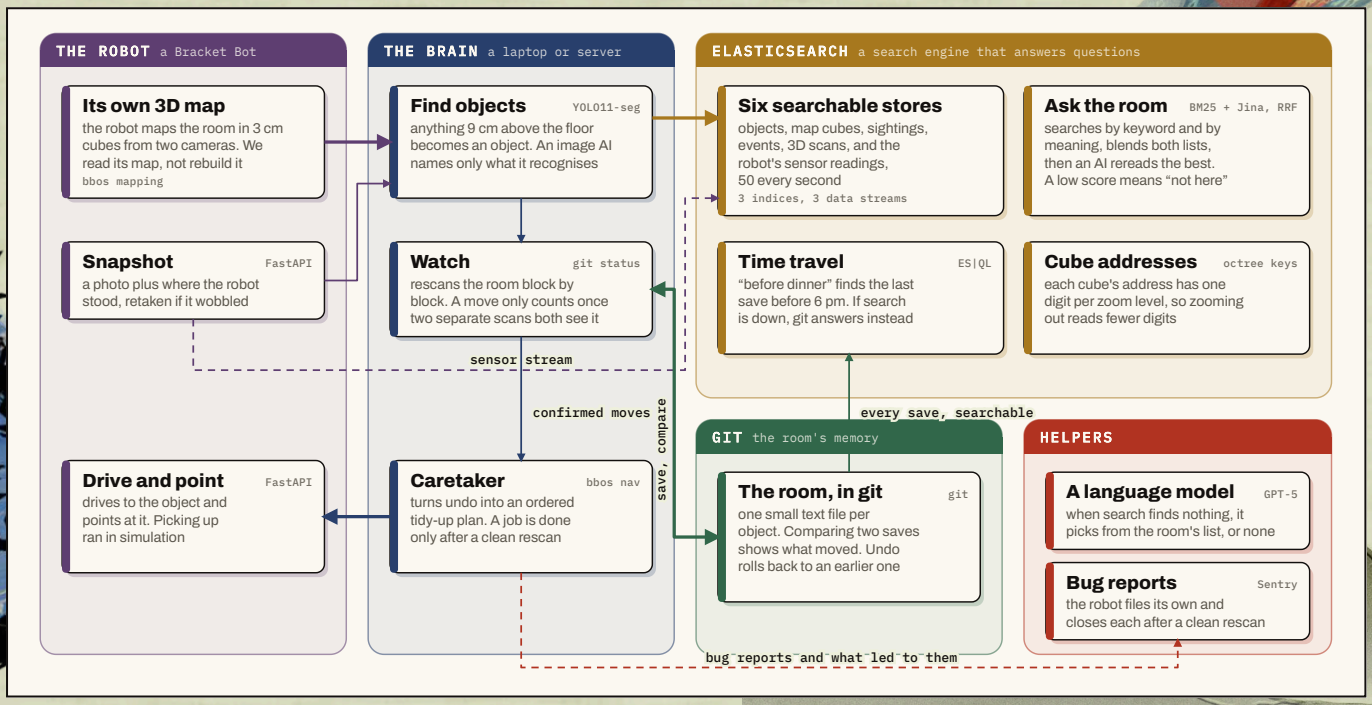
The room lives in a real git project, one small text file per object, so "what moved?" is a side-by-side comparison of two saves, and undo is rolling back to an earlier one. **Git writes the room down. Elasticsearch, a search engine, answers questions about it.**

HACKATHON PAGE
devpost.com/software/house-bot

GITHUB
github.com/DanielWLiu07/hack-the-north

LIVE
gitirl.health

DEMO
youtu.be/FEGR3mJmXMg



6.9M
robot sensor readings, all added up in 0.03 seconds

6
searchable stores, every entry linked to Sentry

60%
of a camera photo must line up with the robot's map, or it names nothing

2
separate scans must both see a move before it counts

SEARCH

Keywords and meaning, blended

Ask for "the thing I cut paper with" and Elasticsearch runs two searches at once. One matches keywords. The other matches meaning, with an AI that turns words into numbers so similar ideas sit close. A second AI rereads the blended top 50 and finds the scissors.

NOT FOUND

Teaching search to say no

Meaning search always returns its closest match, even for things the room does not have, so I measured a minimum match score. Below it the answer is "not here". Just above it, the robot asks what you meant. "Pick up the trash" is refused instead of grabbing a cup.

FILING

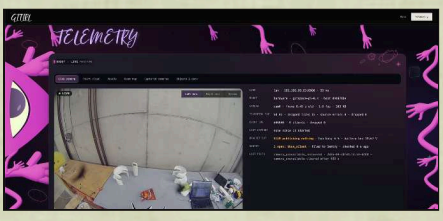
An address that zooms

Each 3 cm cube gets an address of digits, one for each time the room is split in half, so zooming out just means reading fewer digits. As each cube arrives, Elasticsearch files it under the bigger regions it sits in.

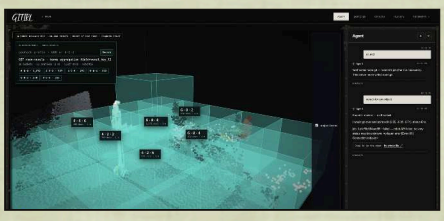
TRACING

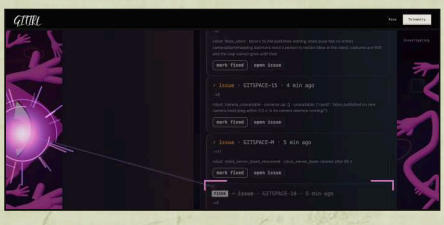
From a bug to the moment behind it

Every record is stamped with an ID from Sentry, a service that collects the software's bugs. So from any failure, one search finds the scan behind it and the robot's sensor readings in the seconds before.



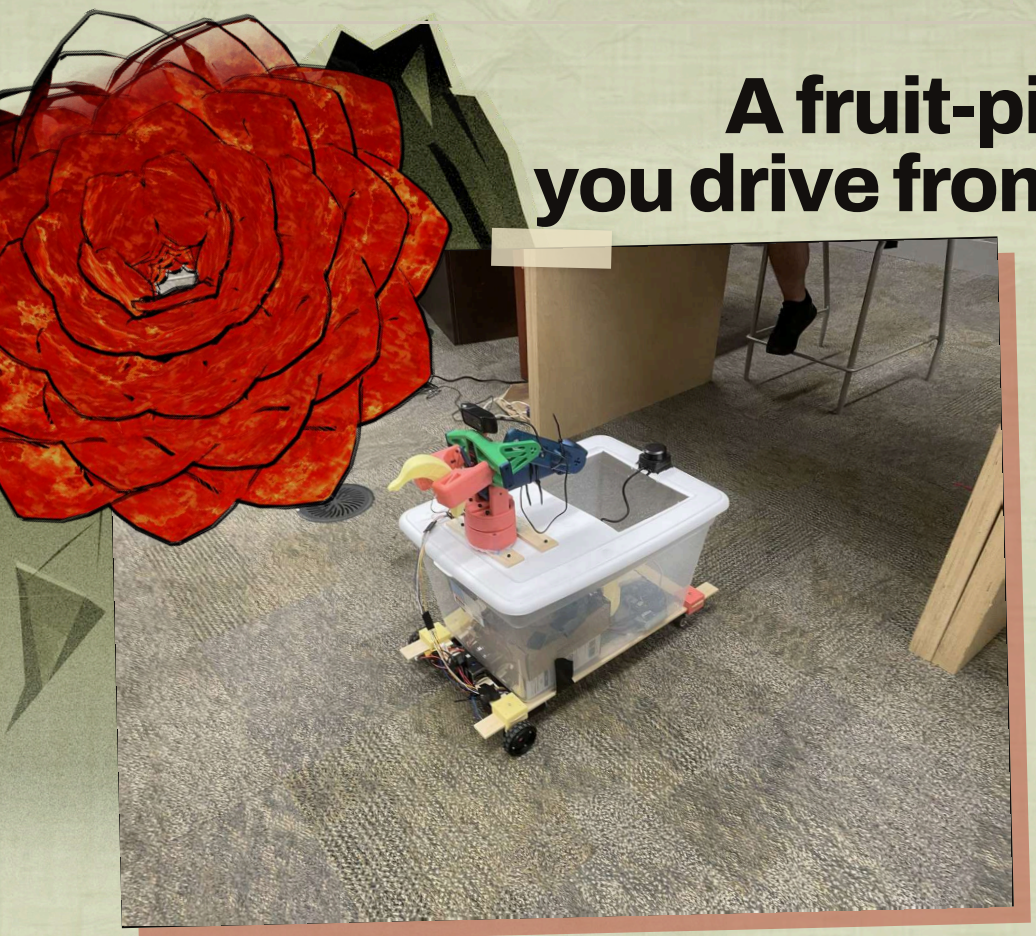
LIVE SENSOR READINGS


ASK THE ROOM A QUESTION


BUG REPORTS THE ROBOT FILED ITSELF

LINES OF CODE **38,500** BUILD TIME **36 HRS** TEAM **4 PEOPLE**

A fruit-picking robot you drive from a browser.



POMME, ARM UP, READY TO PICK

It finds fruit, decides if it is ripe, picks it and sorts it. One chip, a small computer, does the seeing and deciding. A second, simpler chip runs the wheels, arm and brakes and reacts instantly, so the **emergency stop still works** even if the first chip crashes. At the event we drove it from a browser tab. Try a simulated Pomme at **pomme.live**

AWARD
Qualcomm's Build at the Edge prize

MADE AT
Hack the 6ix 2026

TEAM
Four

UNDER THE HOOD

SEEING

Ripeness is colour

I trained the fruit detector on 18,000 computer-drawn pictures coloured like our 3D-printed fruit. One look names the fruit and whether it is ripe. Training usually tints pictures at random, but I kept the tint near zero, because here ripeness literally is colour.

DECIDING

Checked 20 times a second

Roam, spot a fruit, drive up until it fills a tenth of the camera's view, centre it, pick, sort, drop. The plan rechecks its step 20 times a second and never waits on the arm, so it can react mid-move.

MAPPING

A map from a spinning laser

I wrote the mapping. Lidar, a spinning laser that measures distance, sweeps the room, and the robot tracks itself by lining up each sweep with the last. My first version decided a sensor sitting still had wandered 300 metres, so the rewrite ignores tiny wobbles and impossible jumps.



THE PRINTED ARM, IN PA

THE PRINTED ARM

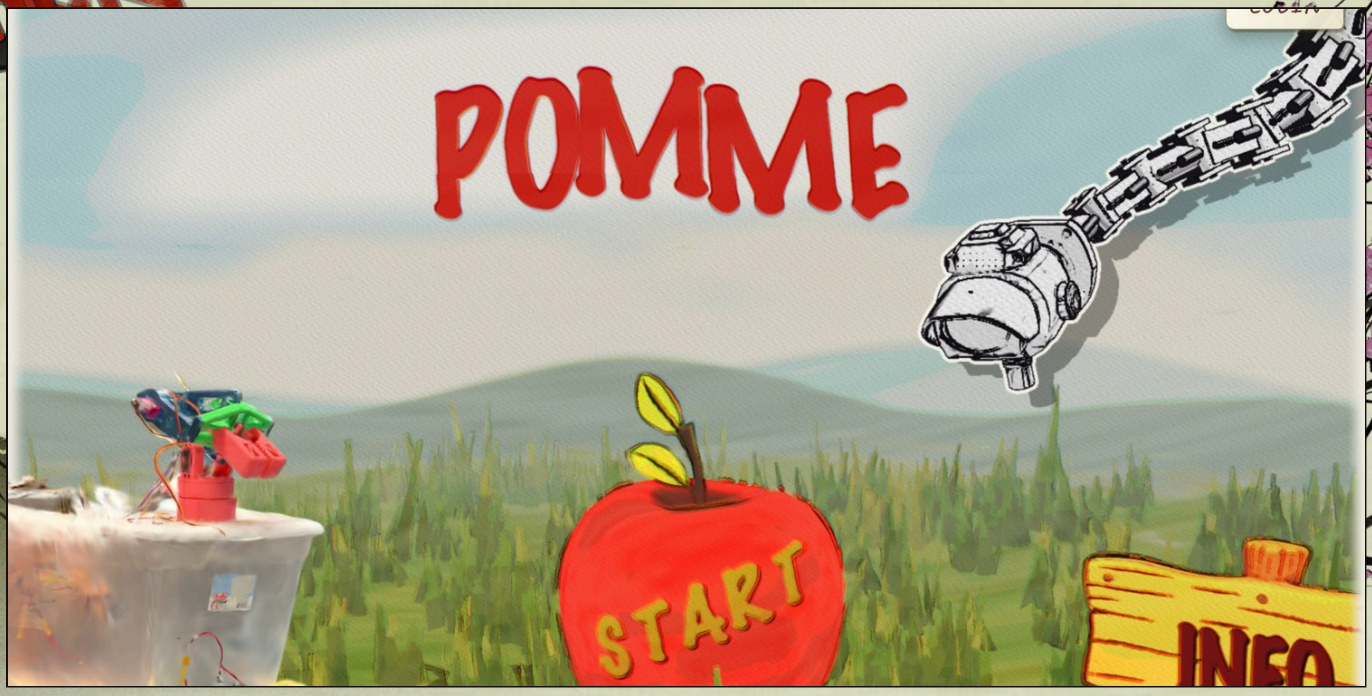
Open a tab. Drive a robot.

The website is the cockpit. A live map, the robot's camera view, every pick counted, and a PlayStation controller **read straight from the browser**. No app to install. The public site runs a simulated robot, so anyone can try it.

HACKATHON PAGE
devpost.com/software/autoagrinator

GITHUB
github.com/DanielWLiu07/pomme

LIVE
pomme.live



9
pages in the dashboard

5
status updates a second from the robot

30
steering updates a second from the controller

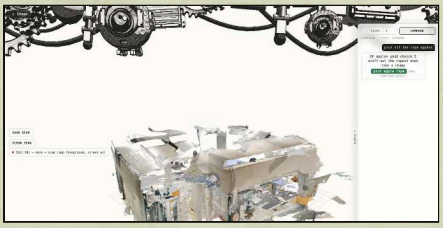
3.2 MB
fruit detector running in your browser, shrunk from 11.6 MB

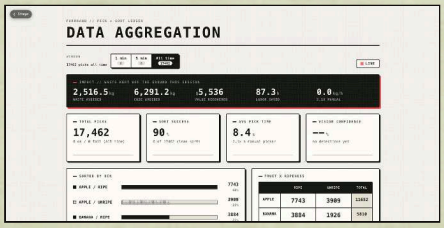
LANDING
A painted orchard, a real robot
The rover rolling in is a Gaussian splat, a 3D capture rebuilt from ordinary phone video of the real robot, set in an orchard scene filtered to look hand-painted, like this book.

THE ROOM
Click a TV, tune in
The dashboard is a dark 3D room of old TVs, and each TV is one page. Click a TV, its screen fills with static, the view dives into it, and that page tunes in behind the static.

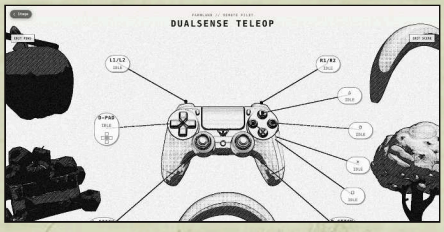
IN YOUR BROWSER
The detector, shrunk
Storing the detector's numbers with fewer digits made it a quarter of the size and 2.6 times faster, but the small copy lost its sense of how sure it is, which the robot needs. So it labels every photo in the website's harvest log, and the full-size detector stays on the robot.

DRIVING
Hold the trigger to move
A PlayStation controller works straight from the tab. Steering goes out 30 times a second, and the wheels only move while you hold the right trigger. Steering goes straight to the robot, skipping the hub, the server that relays every other message.


WHAT THE ROBOT SEES, LIVE



EVERY PICK, COUNTED



DRIVE IT WITH A CONTROLLER

Two brains, one robot.

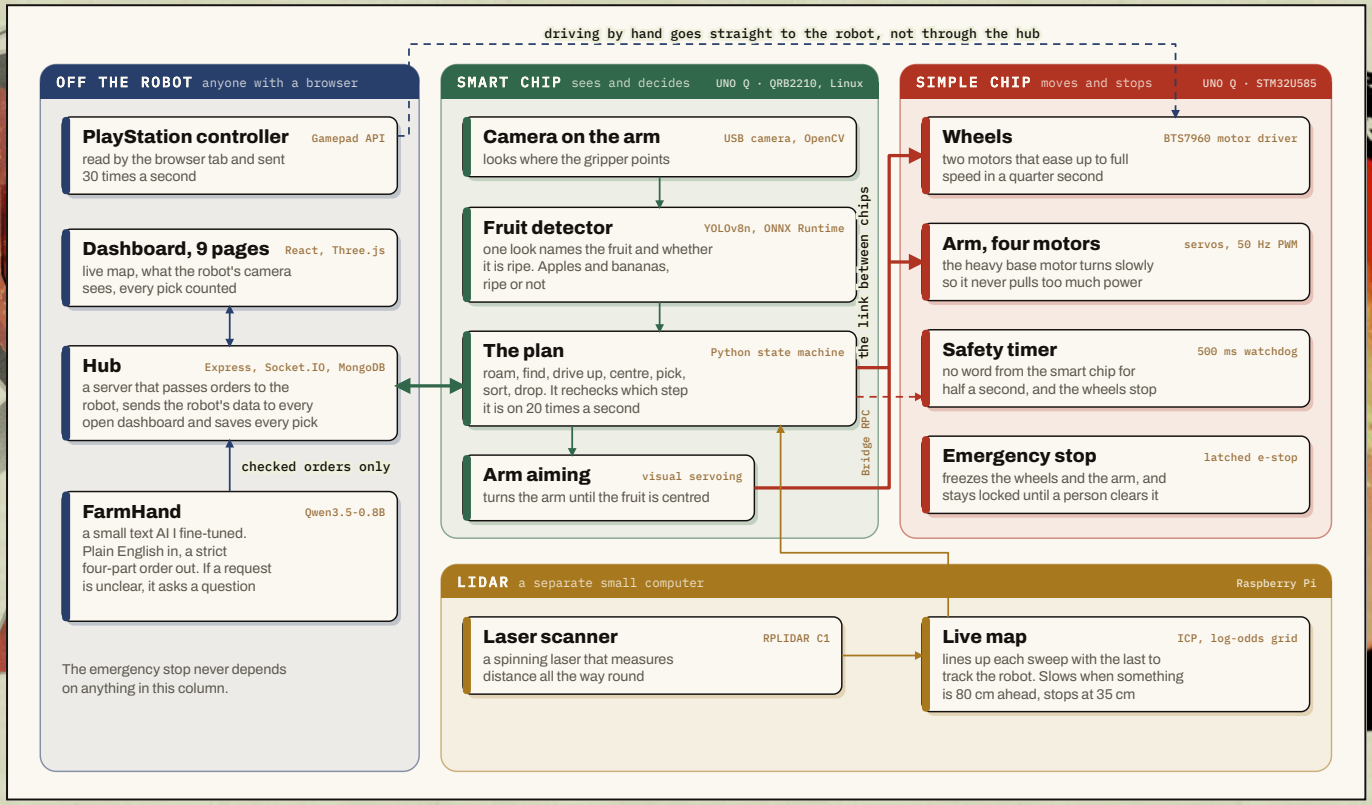
The board has two chips. The smart one, a small computer, sees the fruit and decides what to do. The simple one moves the motors and owns the brakes, so **if the smart chip crashes, the robot still stops**.

THINKS
20 times a second

ARM
Four motors

MAP
From a spinning laser

SIGNAL LOST
Wheels stop in half a second



SAFETY

The brakes answer to no one

If the smart chip stops saying "still here" for half a second, the simple chip cuts the wheels by itself. An emergency stop also freezes the arm until a person clears it.

LANGUAGE

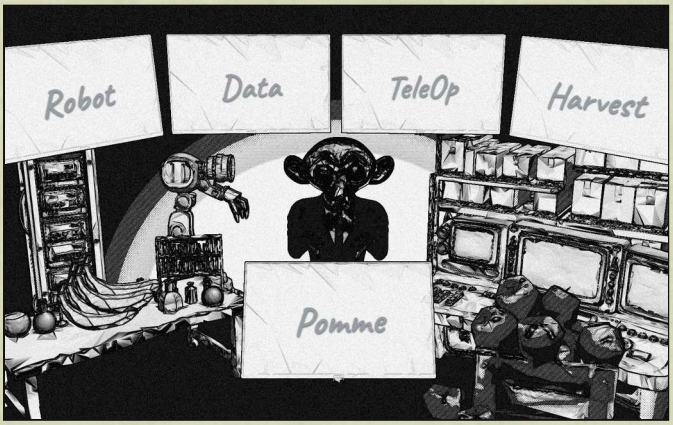
FarmHand, my small text AI

FarmHand turns requests like "grab every ripe banana" into robot orders. I trained it on 2,349 computer-made commands. Tested on 58 new commands full of typos and slang, it got 81% right. Hand-written rules got 47%.

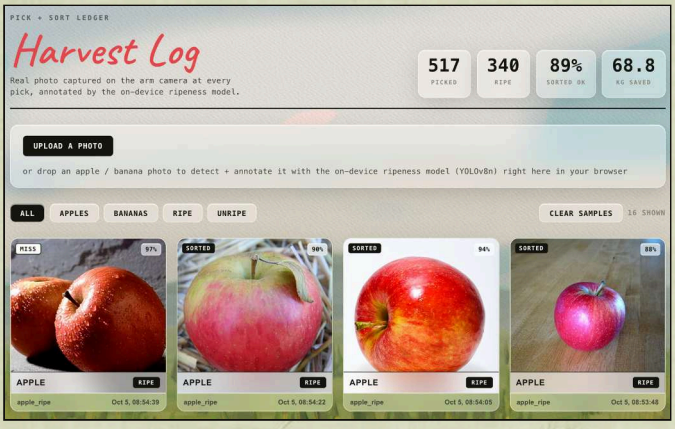
GUARDRAIL

Only strict orders get through

FarmHand's reply must be a four-part order naming the task, fruit, ripeness and direction. Anything else is thrown away before it reaches the robot. Unclear requests get a question back, not a guess.



THE TV ROOM, ONE SCREEN PER PAGE



THE HARVEST LOG, LABELLED RIPE OR NOT IN YOUR BROWSER

LINES OF CODE **51,300** BUILD TIME **10 MONTHS** TEAM **SOLO**

My personal website, one look per page.

danielwliu.com is where I keep my work. Instead of a template, every page is its own little world, built from scenes I made in Blender, free 3D modelling software, and the tour ends at my resume, **dealt to you across a poker table**.

BUILT **Solo** LIVE **danielwliu.com** CODE **GitHub**

1 LANDING | 2 ABOUT | 3 EXPERIENCE | 4 PROJECTS | 5 RESUME



1 · THE LANDING, IN THE SAME INK AS THIS BOOK

1 · LANDING

An ink painting comes alive, with trees sweeping in from both sides and a glowing sun. Before any of it loads, **you pick your quality**. Cinematic plays the video and animation on a fast computer. Lite shows still images that load instantly on anything.

2, 3, 4 · EVERY PAGE ITS OWN SET

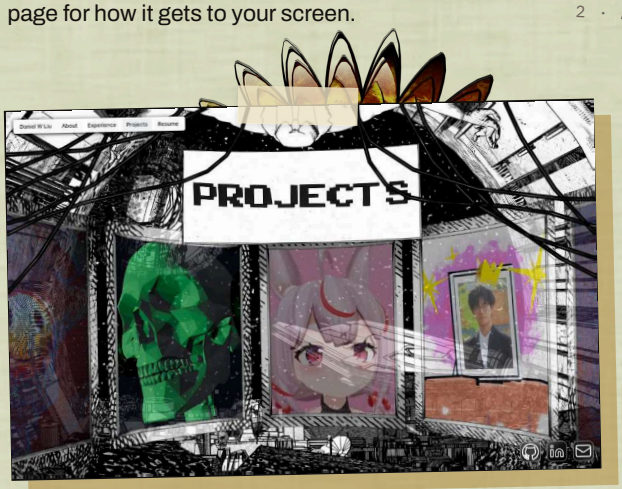
About paints itself in like wet watercolour. Experience lists my jobs in front of a tower of 3D billboards. Projects opens with a manga-style film, then the projects play as videos on a carousel.

The finale is the resume, a 3D casino where you can roll the dice and knock the chips around. Turn the page for how it gets to your screen.

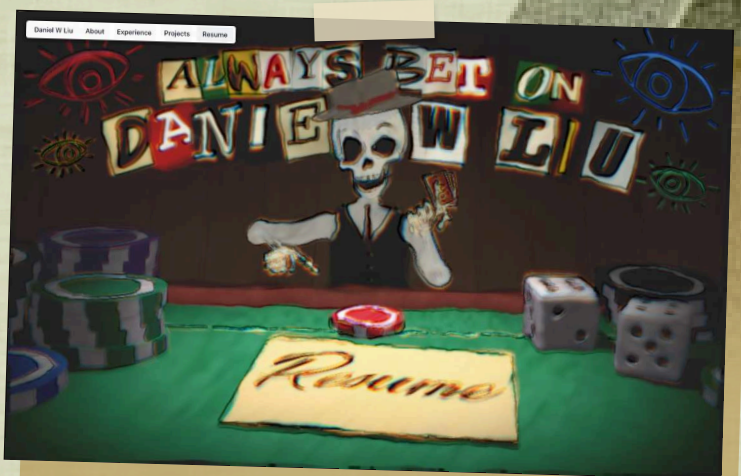


2 · ABOUT

3 · EXPERIENCE



4 · PROJECTS, A VIDEO CAROUSEL IN A MANGA ROOM



5 · THE RESUME, DEALT TO YOU AS A CARD BY A SKELETON DEALER

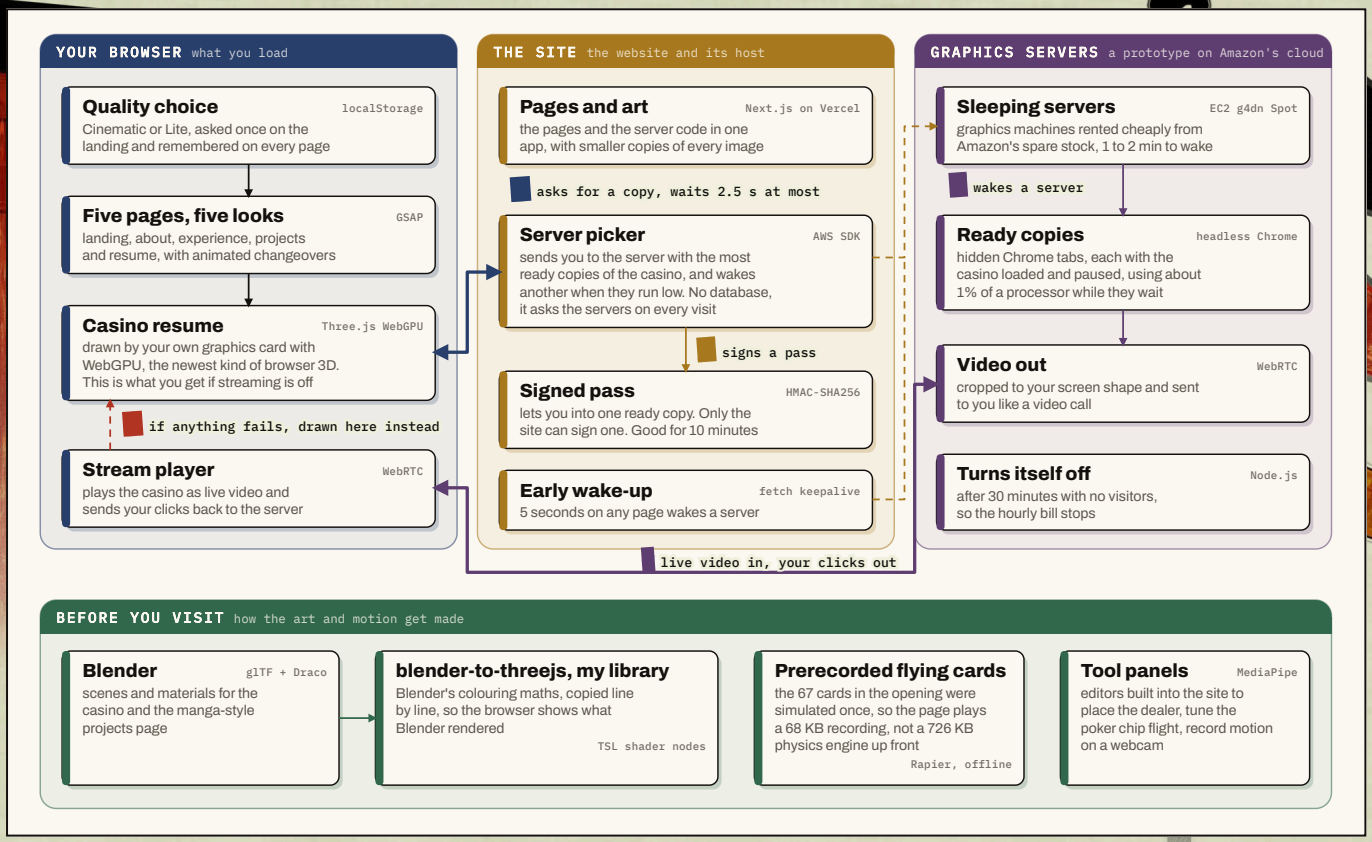
How the casino gets to you.

The resume is a 3D casino, and not every laptop can draw it. So I built a way to start by watching it as live video from a graphics server, the way cloud gaming works, while your own computer, if it is strong enough, gets ready behind it and takes over before the dealer's opening starts. It is a working prototype, off on the live site until I flip one setting. If anything about streaming fails, you get the normal page. **Streaming can only make the page faster, never break it, whatever computer you are on.**

GITHUB
github.com/DanielWLiu07/daniel-w-liu-portfolio-website

MY BLENDER-TO-BROWSER LIBRARY
github.com/DanielWLiu07/blender-to-threejs

BUILT
Solo



0.3 s
from connecting to the first streamed picture, server on my laptop

0.94 → 0.18 s
the longest freeze while the casino loads, before and after my fixes

0
databases. The site asks the servers directly which one has room

30 min
with no visitors, and a server turns itself off so the bill stops

SHADERS

Blender, matched exactly

Three.js, the browser's 3D toolkit, does some colour maths differently from Blender, like dividing by zero or blending two colours, so a scene can load fine and still look wrong. My library, blender-to-threejs, rewrites each of Blender's colouring steps from Blender's own source code so the two match.

STREAMING

A copy that is already running

Five seconds on any page wakes a graphics server in the cloud, which takes one to two minutes. By the time you reach the resume, a copy of the casino is loaded and waiting. Your pass to it is signed, so it cannot be forged, and expires after 10 minutes.

FALLBACKS

It can only make things faster

No free copy, a server that takes over 2.5 seconds to answer or a dropped video all end the same way. The casino is then drawn by your own computer's graphics chip, like any 3D web page.

BUILT WITH `Next.js for the website` `Three.js for 3D in the browser` `WebRTC for live video` `Amazon cloud servers` `MediaPipe for webcam tracking` `Blender`

LINES OF CODE **28,900** BUILD TIME **4.5 MONTHS** TEAM **SOLO**

A world made of cubes, in four dimensions.

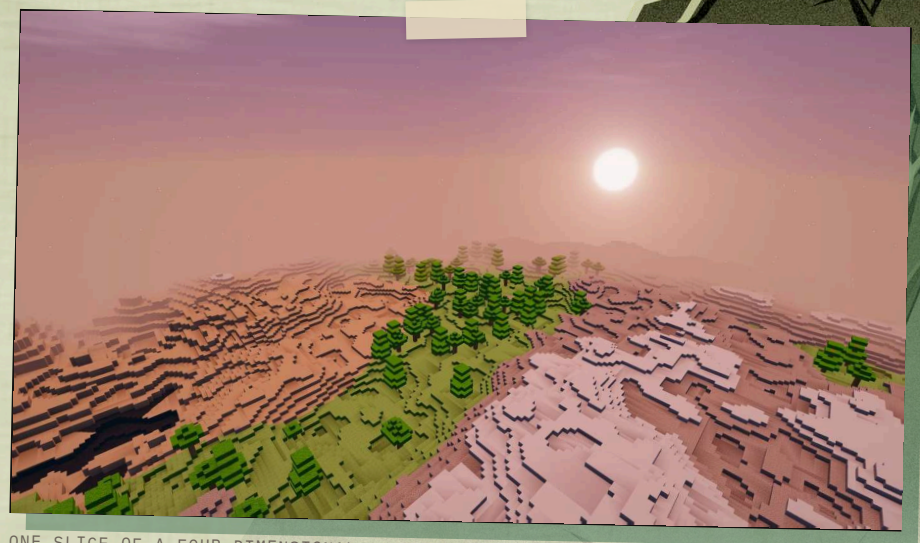
The world has four directions, not three. What you walk around in is a 3D slice of it, and scrolling turns the slice. The camera never moves, yet the landscape rebuilds itself, because **you are looking at a different slice of the same world**.

BUILT
Solo, from scratch

STARTED
May 2026

WRITTEN IN
C++, with OpenGL for graphics

EVERY BLOCK
A cube, or a slice of a 4D cube



ONE SLICE OF A FOUR-DIMENSIONAL WORLD

SAME WORLD, SAME CAMERA.



FLAT SLICE

TILTED 4°

TILTED 9°

TURNING

A sixth of a degree per click

Each click of the scroll wheel turns the slice about a sixth of a degree, pivoting on you, so the ground under your feet stays put. Holding E or Q moves you along the fourth direction, and a lake with a treeline gives way to a snowfield.

TERRAIN

A 4D hill formula, written by hand

The library I used to shape hills only works in three dimensions, so I wrote my own 4D version. Each point blends random slopes from the 16 corners of a tesseract, a 4D cube. Near the surface, a second 4D formula carves overhangs.

SKY

Nobody places the rainbow

When it rains, the sky's drawing code puts the rainbow 42 degrees from the point opposite the sun, like real light does, so it rises by itself at sunset. 9,000 fireflies each work out their own flight path on the graphics chip.



RAINBOW



AURORA



FIREFLIES



UNDERWATER

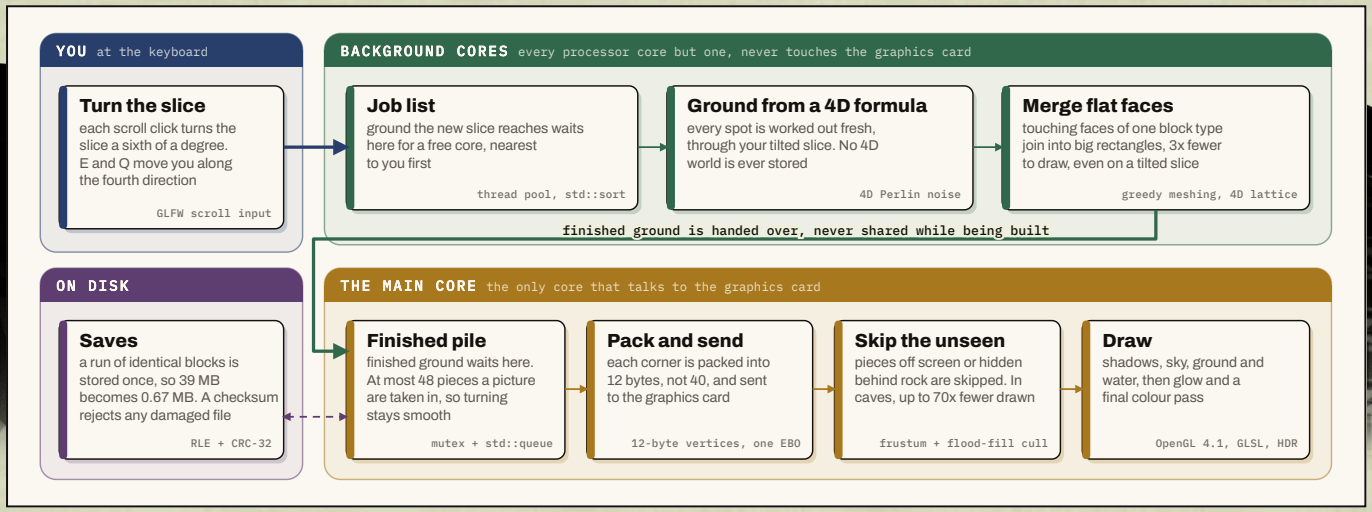
How a 4D world gets drawn.

Nothing four-dimensional is stored. The world is a formula, and every patch of ground is worked out fresh for the angle of your slice. Most of the processor's cores build ground while one core draws, and touching faces get merged so there is far less to draw, **even when the slice is tilted**, which the engine's own design notes once said could not be done.

GITHUB
github.com/DanielWLiu07/voxel-engine

TESTED ON
Mac and Linux, on every change

BUILT
Solo



0.008 s
to draw each picture of a 277-million-block world on my laptop, in its ordinary 3D mode

11.6x
less graphics memory for the whole world, 126.7 MB down to 10.9 MB

3x
fewer faces to draw, once touching faces are merged

58x
smaller saves, 39 MB to 0.67 MB

SHAPES

A block is a slice, not a cube

Turn the slice in one direction and every block stays four-sided. Turn it in two at once and triangles, pentagons and hexagons appear. Leaving height aside, a 4D cube has six walls that can cross the slice, so a block is never more than a hexagonal pillar, a six-sided column.

MERGING

Big flat faces, even on a tilt

Touching faces of the same block type merge into big rectangles, so the graphics card draws 3x fewer. On a tilted slice the faces stop lining up on screen, so I merge them in the 4D grid, where they still do.

TEAMWORK

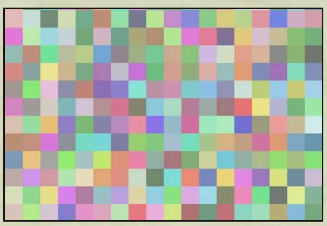
Many builders, one painter

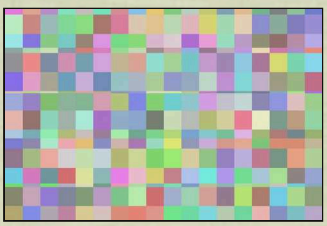
A processor has several cores, small brains that work at once. All but one build ground in the background, each from its own copy of the neighbouring ground, then hand it over. Only the last core talks to the graphics card, so they never fight over it.

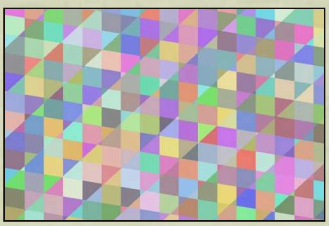
TESTS AND SAVES

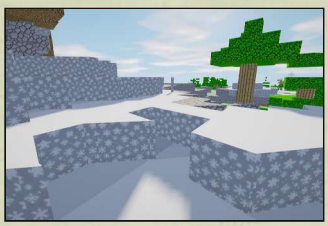
Turn it, turn it back, check every block

Turn the slice 14 degrees and back, and every block must return exactly as it was. That check and 19 others run automatically every time I upload new code. Saves store "stone, forty in a row" instead of forty stones, and a built-in error check rejects corrupted files.


FLAT SLICE, SQUARE BLOCKS


TURNED ONE WAY, FOUR SIDES


TURNED TWO WAYS, HEXAGONS


A BLOCK AS ITS TRUE SLICE

LINES OF CODE **12,500** BUILD TIME **WEEKEND + 1 MONTH** TEAM **4 PEOPLE**

It guesses where you are from a photo.



THE LANDING PAGE

Drop in a picture of anywhere and it works out where on earth it was taken, the way a geography nerd would, from road lines, scripts on signs, licence plates, plants and the shape of the hills. Then it **shows its working**, so you learn the clues too.

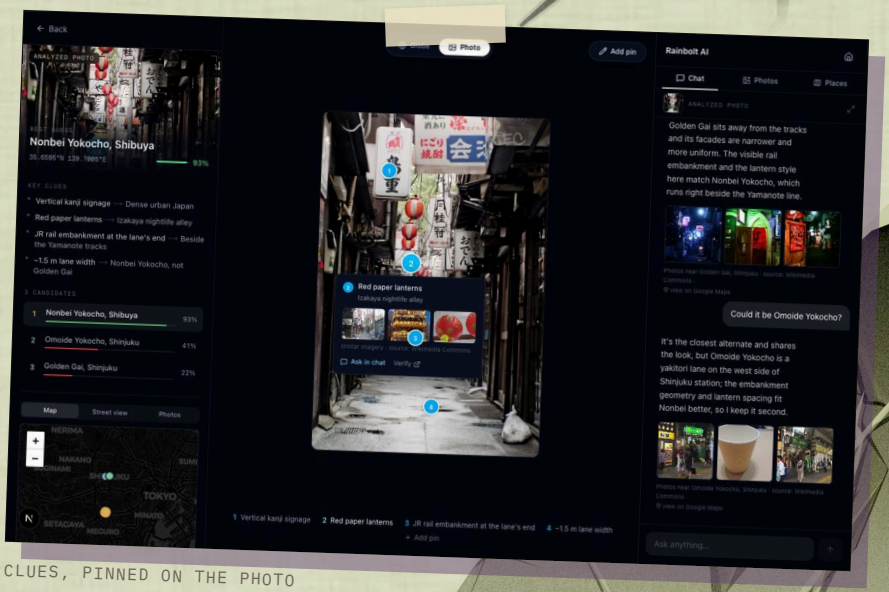
AWARD
Best UI Hack

OUT OF
124 projects

TEAM
Four

WHAT I BUILT

- **The 3D globe.** About 156,000 glowing dots, drawn by graphics code I wrote, with a denser layer that fades in as you zoom. Up close each dot doubles in size, so coastlines read as solid land.
- **Clue pins.** After the guess, another AI pass rereads your photo and pins 3 to 6 numbered clues on it, each with example photos from Wikimedia.
- **A chat that does not lose its place.** If Google's AI hits its daily limit, a lighter backup AI answers. If a reply cuts off, the chat keeps what was already written.
- **The relaunch.** Its library of location-tagged photos was deleted, so I rebuilt it from a public set, up to the free plan's monthly upload cap, and rewrote the 242 clues. Sign-in went from two services to one, plus a guest mode and tutorial.



CLUES, PINNED ON THE PHOTO

WHY IT IS A GAME

Guessing places from a photo is already a popular online game, GeoGuessr. Rainbolt.ai plays it with you and explains every move, so **each clue it spots is one you learn to spot yourself**. By the next photo, you might get there before it does.

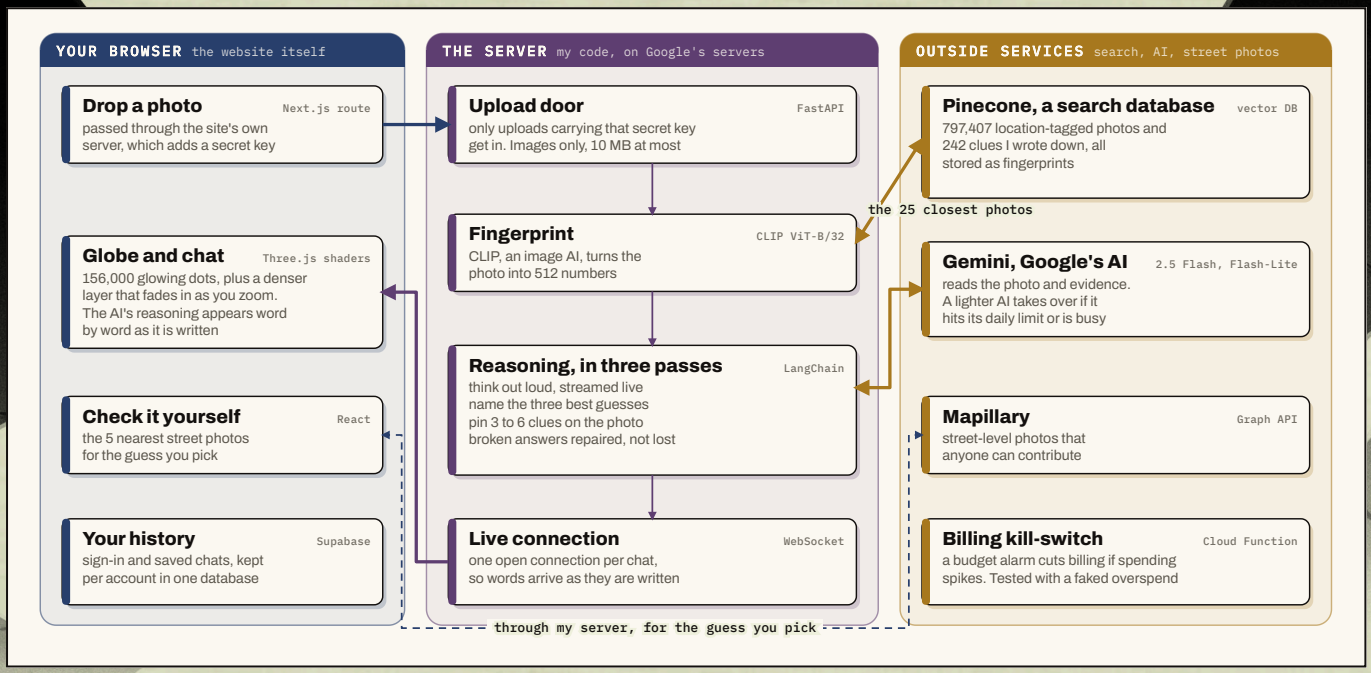
One photo in, one pin out.

Your photo becomes a fingerprint of 512 numbers, matched against 797,407 location-tagged photos and 242 written clues. An AI reasons through the evidence out loud, places its guesses on the globe, and **nearby street photos let you check each guess yourself**.

HACKATHON PAGE
devpost.com/software/rainbolt-ai

GITHUB
github.com/dtpu/rainbolt.ai

LIVE
rainboltai.vercel.app



797,407
location-tagged photos in the library I rebuilt

242
location clues, from licence plates to road posts

1st
Best UI (user interface), out of 124 projects

3
AI passes per photo, to reason, guess three places and pin clues

MATCHING

Close fingerprints, close places

CLIP, a free image AI, runs on my server and turns your photo into 512 numbers. Pinecone, a search database, returns the 25 closest photos and drops weak matches. I never average their locations. The AI weighs each match's place and score itself.

CLUES

Pictures compared with words

Each clue is a sentence, like the shape of a country's bollards, the short posts beside the road. Similarity runs from 0 to 1. A good photo-to-photo match scores about 0.7, but a photo-to-sentence match only about 0.2, so clue matches get their own, much lower cut-off of 0.22.

REASONING

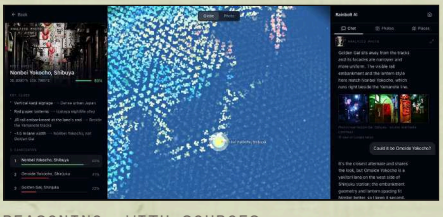
Answers that survive bad replies

The three guesses should come back in a strict data format, but the AI sometimes breaks it. So my code tries a strict reader, then a forgiving one, then a tool that repairs missing commas and brackets. A stray apostrophe no longer loses a guess.

SAFETY

A bill that cannot run away

The server only accepts photos sent through my website, which attaches a secret key visitors never see. If spending spikes, a Google budget alarm triggers a small cloud program that switches billing off. I tested it by faking an overspend.


REASONING, WITH SOURCES


GUESSES ON THE GLOBE


EVERY PHOTO YOU HAVE TRIED

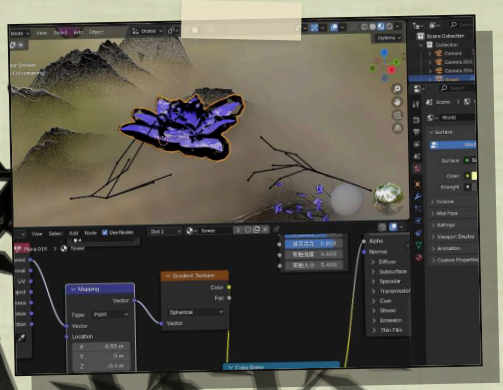
WORKS 6 BUILD TIME ONGOING TEAM SOLO

Made for fun, to learn.

No brief, no client, no deadline. I wanted to see if I could, and then I wanted to see if I could **do it better**.

BRIEF **None** MADE IN **Blender and Unity**

IN BLENDER scenes, a walking goose and two drawing styles



The ink landscape

The scene behind this book's cover: mountains, plum blossom and a red moon, rendered to look brushed in ink. Here it is open in Blender, with the web of colour nodes that makes the 3D look painted.



A goose that walks

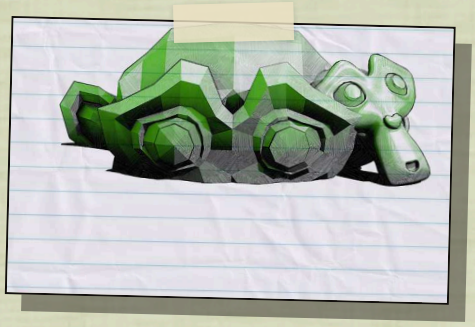
A free goose model by the artist stickbone that I rigged in Blender, giving it a skeleton so it can move. On my website you walk it around a village green, feet planted on the ground, and every honk pops comic squiggles out of its bill.



A website you walk into

The webring for Waterloo's Computing and Financial Management program, one site linking every student's personal website. I made its scenes in Blender, used as the site's backgrounds and its opening film.

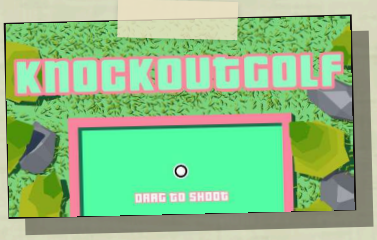
uwaterloocfm.com



Pencil on notebook paper

A second drawing style, nothing like the ink. Any model can be drawn as pencil shading on creased, lined paper. Here it is on Suzanne, Blender's test monkey.

IN UNITY two games, in a free game engine



Knockout Golf

A same-room party game where you are the golf ball: drag back, aim, launch. No waiting between shots, so everyone is always crashing into each other. Knock them off the stage or race them to the hole. Models from Blender, swaying grass from Unity's Shader Graph.

GitHub



Labyrinth of Doom

Online multiplayer in the spirit of Among Us. Everyone gets a torch in a pitch-dark maze, except the Minotaur, who sees in the dark and hears you walk. Light six statues to put it to sleep, but each one shows it where you are. A two-week game jam, a build-it-fast contest, for school. Stack Overflow and YouTube tutorials carried me.

itch.io

I build things for the love of the game.

More at danielwliu.com.

NAME	DANIEL W LIU
EMAIL	D347LIU@UWATERLOO.CA
SITE	DANIELWLIU.COM
CODE	GITHUB.COM/DANIELWLIU07
LINKEDIN	LINKEDIN.COM/IN/DANIELWLIU

